Guía del estudiante: diagnóstico técnico y refactorización

Ingeniería de Software II | Trabajo por equipos | 2 sesiones de 2 horas

Propósito. Analizar una funcionalidad ya implementada, identificar hallazgos reales, mejorar su diseño interno sin alterar el comportamiento esperado y dejar trazabilidad técnica en el repositorio.

1. Reglas de la actividad

• Seleccionar una funcionalidad que involucre una transacción de negocio y, de ser posible, varias entidades.

• Documentar al menos tres hallazgos verificables; uno como mínimo sobre arquitectura, SOLID, responsabilidades, dependencias o separación de capas.

• Clasificar los hallazgos como defecto funcional, deuda técnica, riesgo arquitectónico u oportunidad de mejora. No todas las observaciones obligan a modificar código.

• Priorizar los hallazgos y refactorizar al menos uno, conservando el comportamiento observable. Una corrección de defecto debe declararse como tal.

• Usar IA solo como apoyo: verificar manualmente cada afirmación, ubicación y consecuencia. No subir secretos, credenciales ni datos personales a servicios de IA.

2. Secuencia de trabajo

Momento	Actividades	Resultado
Sesión 1 · 20 min	Seleccionar funcionalidad y recuperar requisitos y arquitectura.	Alcance delimitado
Sesión 1 · 40 min	Rastrear controladores, servicios, entidades, persistencia y pruebas.	Mapa de componentes
Sesión 1 · 40 min	Inspección asistida y verificación manual; registrar 3 hallazgos.	Diagnóstico con evidencias
Sesión 1 · 20 min	Clasificar, priorizar y realizar commit documental.	Diagnóstico versionado
Sesión 2 · 55 min	Preparar comprobaciones, implementar un cambio acotado.	Código refactorizado
Sesión 2 · 35 min	Ejecutar pruebas; comparar antes y después.	Evidencia de conservación
Sesión 2 · 30 min	Completar informe, commits y sustentación.	Entrega y defensa

3. Lista de inspección técnica

Zona	Preguntas orientadoras
Controladores	¿Contienen decisiones del dominio, cálculos o acceso directo a persistencia?
Servicios	¿Hay responsabilidades independientes mezcladas? ¿Dependencias innecesarias?
Dominio	¿Dónde se aplican las invariantes y reglas de negocio?
Código	¿Hay duplicación, nombres ambiguos, condiciones complejas o código muerto?
Arquitectura	¿El código respeta capas, dependencias y decisiones documentadas?
Pruebas	¿Qué verificaciones protegen el comportamiento de la funcionalidad?

4. Prompt de apoyo para IA (copiar y adaptar)

Actúa como revisor técnico de arquitectura y refactorización. Analiza exclusivamente los archivos que te proporcionaré de la funcionalidad [NOMBRE], desarrollada en [TECNOLOGÍA] con arquitectura [DESCRIPCIÓN]. Identifica componentes y responsabilidades. Busca problemas sustentables de separación de capas, SOLID, acoplamiento, cohesión, duplicación y complejidad. Para cada hallazgo entrega: ID; archivo/clase/método y líneas si están disponibles; fragmento de evidencia; explicación; categoría (defecto, deuda técnica, riesgo arquitectónico u oportunidad de mejora); impacto; refactorización posible; y cómo comprobar que se conserva el comportamiento. No inventes archivos, líneas ni problemas. No supongas que un método largo viola SRP. Diferencia hipótesis de hechos. No cambies el código todavía. Señala la información que falta y espera mi validación manual.

5. Artefacto obligatorio en GitHub

Crear docs/refactorizacion/DIAGNOSTICO_REFACTORIZACION.md y una carpeta opcional evidencias/. Debe incluir:

Sección	Contenido mínimo
1. Contexto	Equipo, repositorio, funcionalidad, requisitos relacionados.
2. Arquitectura real	Componentes, dependencias y flujo de la funcionalidad.
3. Hallazgos	3 fichas: ID, ruta, método, líneas, evidencia, categoría, justificación y consecuencia.
4. Priorización	Impacto (1–3), riesgo (1–3), esfuerzo (1–3) y razón de selección.
5. Intervención	Cambio elegido, diseño antes/después y archivos afectados.
6. Verificación	Pruebas ejecutadas, resultados y limitaciones; evidencias antes/después.
7. Cierre	Lecciones, deuda técnica pendiente y vínculo con el proceso de desarrollo.
8. Git	Hashes o enlaces a commits del diagnóstico y la intervención.

6. Plantilla de ficha de hallazgo

ID: H-01 | Tipo: deuda técnica / riesgo / defecto / mejora
Ubicación: ruta, clase, método, líneas | Evidencia: fragmento o referencia exacta
Observación: qué sucede y por qué importa | Criterio: principio/atributo afectado
Impacto y riesgo: argumentos, no etiquetas sin evidencia
Acción propuesta: refactorización concreta | Verificación: prueba o comprobación
Estado: pendiente / seleccionado / implementado / verificado.

7. Evidencia y entrega

• Commit 1 sugerido: docs: registrar diagnostico tecnico.

• Commit 2 sugerido: refactor: separar responsabilidades de [funcionalidad].

• Commit 3 sugerido: docs: documentar verificacion de refactorizacion.

• Entregar enlace al repositorio, ruta del informe, commits y resultados de pruebas. Evitar capturas sin referencias al código.

8. Rúbrica (100 puntos)

Criterio	Puntos
Hallazgos comprobables y evidencia	20
Justificación y clasificación técnica	20
Calidad y alcance de la refactorización	25
Verificación de comportamiento	20
Sustentación y relación con arquitectura	15
Total	100

Pregunta de cierre: ¿Qué práctica del proceso de desarrollo habría permitido detectar estos problemas antes de entregar la aplicación?